

Finding a needle in 4×10^9 configurations

Design-space exploration for compute-near-memory systems

Clément Fournier

ESWEEK 2026

TU Dresden

One question, held for 45 minutes

Every configuration costs a simulation, and there are too many to try. How do you find a good one — and how do you know it is good?

1. Where the configuration space comes from
slides + notebook 1
2. What the space looks like notebook 2
3. Searching it on a budget slides + notebook 3

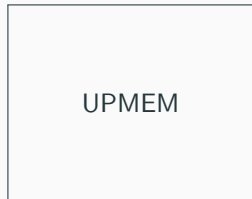
Open the notebooks now

Binder QR / link

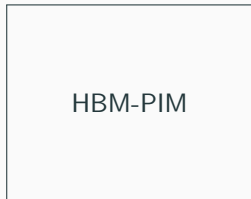
Binder takes a minute to start; let it while we do slides.

The hardware

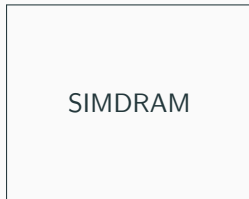
Compute near memory, four ways



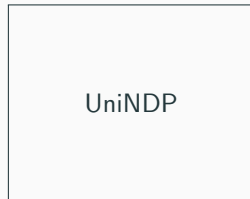
DPU in DRAM ranks



SIMD units per bank



compute in the cell array

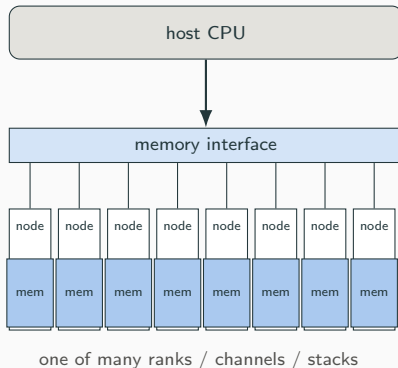


the design space of all of
them

Figures: UPMEM (Devaux, HotChips'19), Samsung HBM-PIM (Lee et al., ISCA'21), SIMDRAM (Hajinazar et al., ASPLOS'21), UniNDP (Li et al., HPCA'25).

What they have in common

- **Many small compute nodes** — thousands, each slow on its own
- **Each owns a slice of memory** — fast to reach locally, slow or impossible across
- **A hierarchy of levels** — bank / rank / channel; node-local scratchpad
- **A narrow interface to the host** — data goes in and out through it
- **Programmable** — what runs where is a software decision



The consequence: a kernel is not *run* on such a system, it is *distributed* onto it. How is the design decision.

The kernel

Our running example: $y = Ax$

$$y_m = \sum_k A_{m,k} x_k$$

for $m \in M$, $k \in K$

Two loop dimensions with very different characters:

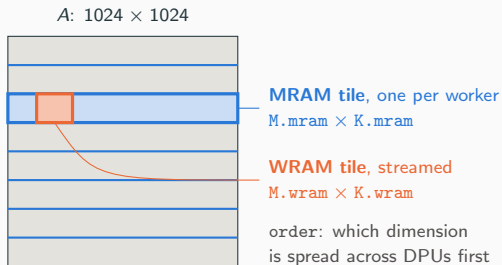
- M is **parallel** — rows are independent
- K is a **reduction** — splitting it means partial sums to combine

In Cinnamon's MLIR input:

```
#upmem = #upmem.platform<type = v1A,  
                                     dpus = 64, tasklets = 16>  
  
func.func @gemv(%A: tensor<1024x1024xi32>,  
               %x: tensor<1024xi32>)  
    attributes {cinm.available_platforms = [#upmem]} {  
        cinm.compute {  
            cinm.op.gemv %A, %x  
        }  
    }
```

One DRAM rank: 64 DPUs, 16 hardware threads each, 64 MB MRAM and 64 KB WRAM per DPU.

Tiling to the memory levels



Each choice must satisfy

- tiles divide the extents: $M.mram \mid 1024$,
 $K.mram \mid 1024$
- inner tiles divide outer ones:
 $M.wram \mid M.mram, \dots$
- one tile per worker:
$$\frac{1024}{M.mram} \cdot \frac{1024}{K.mram} = dpus \cdot tasklets$$
- what the tasklets hold fits: $\leq \text{MRAM}, \leq \text{WRAM}$
- DMA granularity: tiles are multiples of 8 bytes

Every arrow in the picture is a parameter; every “must” is a constraint.

The hardware knobs are not separate

How many DPUs? How many tasklets?

These look like *platform* decisions — fix them first, then tile. But:

- the tile count must equal the worker count:
$$\prod(\text{extent}/\text{tile}) = \text{dpus} \cdot \text{tasklets}$$
- more tasklets $\Rightarrow$ smaller tiles $\Rightarrow$ more launch overhead per byte
- more DPUs $\Rightarrow$ more transfer parallelism, more partial results to gather

Measured on the small space

best latency with tasklets =

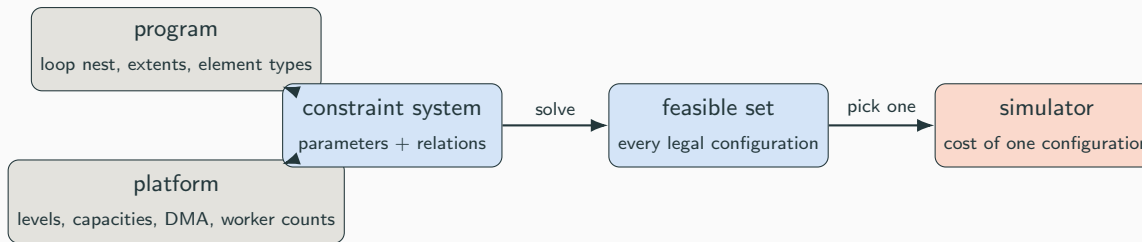
1	2	4	8	16
8.0	4.2	2.4	1.6	7.8

(ms) — “use all the threads” is 5 $\times$ off.

On 4096² with 32 ranks the optimum is **1 tasklet** on all 2048 DPUs. The right knob setting flips with problem size.

Hardware parameters and tiling parameters live in one space.

From structure to a space



1024² **gemv, one rank**

Cartesian product

feasible

density

solved in

4.06×10^9

3 234

8×10^{-7}

10 ms

Nothing in the space was written by hand.

Swap the platform description and the same derivation yields the space of a different CNM system.

Notebook 1

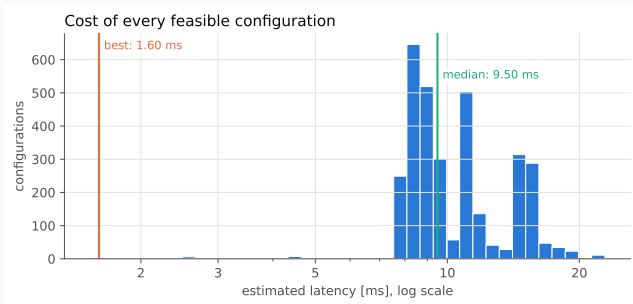
The design space

Pick a configuration by hand and simulate it. Then let Cinnamon derive the space and read its constraints.

 8 min

The landscape

We simulated all 3 234

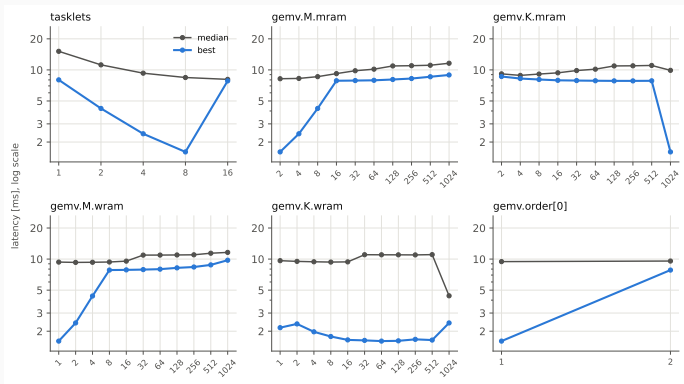


best	1.60 ms
1st percentile	6.25 ms
median	9.50 ms
worst	21.8 ms

within 10% of best	6
within 1.5×	11
within 2×	19

A **needle**, not a hill: the best 0.3% is a separate cluster, 5× below the bulk.

Which parameters matter



tasklets and K.mram decide everything; the WRAM tiles barely move the best line. The ten best configurations differ in *one* parameter.

Notebook 2

The landscape

Where did your guess land? Marginals, the heat map, the top ten. Then pin a hardware knob and search the rest live. 🕒 8 min

The search

Enumerating is not an option

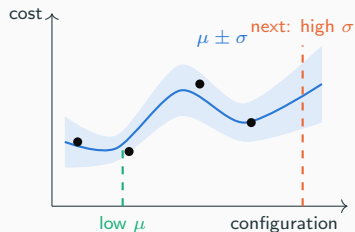
	feasible	one core, fast simulator	
1024^2 gemv, 1 rank	3 234	3 min	the oracle you just used
4096^2 gemv, 32 ranks	26 238	1.5 h	enumerated once, on 14 cores
2048^2 gemm, 32 ranks	2 821 332	47 h	never

Random search is the honest baseline: no model, no assumptions, and with an oracle it costs nothing to evaluate — draw from the pool.

Model-based search spends simulations where a model says they are worth spending. The question is whether the model learns the landscape faster than random sampling stumbles on it.

Bayesian optimisation in one slide

1. **Initial sample** — a few spread-out configurations, simulated
2. **Surrogate** — fit a model to what was seen; for any configuration it predicts a cost μ and an uncertainty σ
classically a Gaussian process; here an ensemble of small networks
3. **Acquisition** — score candidates by $\mu - \kappa\sigma$: low predicted cost *or* high uncertainty is attractive
4. **Simulate** the best-scoring candidate, add it, go to 2



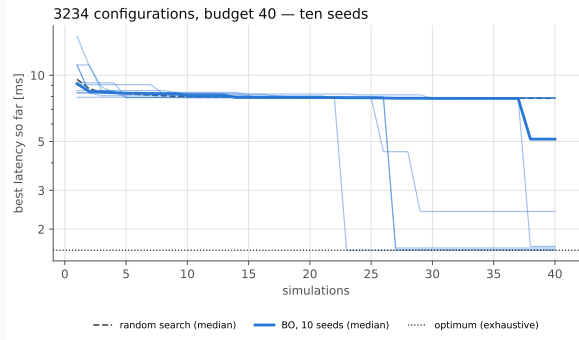
The infeasible configurations never reach the surrogate: the constraint solver hands it the feasible set, so every simulation is spent on something that can run.

Notebook 3

The search

Forty simulations, then five seeds, then a hundred — each scored against the oracle. Then the space of 26 238. 🕒 9 min

What a budget buys

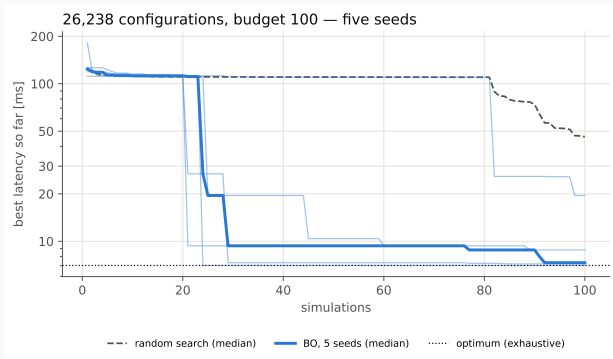


Small space, budget 40, ten seeds

seed	best [ms]	rank
166	1.60	1
197	1.61	2
104	1.64	4
290	1.67	6
135	2.41	11
73	7.85	45
259	7.87	66
228	7.89	80
42	7.90	103
321	7.91	107

Four of ten find the fast line; random search has a 14% chance. With **budget 100**, four of five seeds land on the exact optimum.

Where enumeration was never an option



26 238 configurations, budget 100

	ms	rank
optimum (oracle)	7.0	1
Bayesian, best of 5 seeds	7.0	1
Bayesian, median seed	7.3	8
Bayesian, worst seed	19.5	82
random search, median	46	
median configuration	125	

Three of five within 10 % of an optimum that only 10 of 26 238 configurations come within 10 % of — with 0.4 % of the simulations.

The 2.8-million space: 60 simulations in 14 s. No oracle — the two spaces above are its calibration.

Take-aways

What carries over

1. **Derive the space, don't write it.** Program structure $\times$ platform structure $\rightarrow$ constraints $\rightarrow$ feasible set. The same machinery, another platform description, another system.
2. **Hardware knobs live in the same space as the tiling.** Pinning them first caps what any tiling can reach — and the right pin flips with problem size.
3. **Look at the landscape before trusting a search.** Needle or hill, which parameters move the best line, how many configurations are within 10% — an exhaustive run on a *small* instance answers all three.
4. **Score searches against an oracle where you can afford one.** A single run is an anecdote; seeds give a distribution; the oracle gives a regret.
5. **Then spend the budget where you cannot.**

Cinnamon: github.com/tud-ccc/Cinnamon this tutorial: [repository]